

Motor de Mineração e Contexto (Backend)

A arquitetura base utiliza Node.js e Python (FastAPI) para garantir resiliência e processamento assíncrono seguro.

- **Scraping Georreferenciado:** Integração com APIs de mapas para extrair negócios locais, aplicando validação estrita de dados com schemas no backend antes de qualquer processamento ou persistência.
- **Filtros de Qualificação Ativa:** Algoritmos para descartar automaticamente contatos sem celular válido ou que já possuam infraestrutura concorrente identificável.
- **Ingestão de Repositórios:** O sistema lê arquivos estratégicos do código-fonte para que o LLM deduza o Perfil de Cliente Ideal (ICP) e argumentos de quebra de objeções.
- **Processamento Físico Isolado:** Fila de disparos com controle de concorrência estrito (1 requisição por vez) para gerar os áudios locais sem estourar o limite de memória da placa de vídeo.

Engenharia Frontend e UX (Next.js)

A interface do CRM adota o modelo App Router, focando em performance de renderização e ergonomia visual avançada.

- **Server Components Padrão:** Separação estrita entre lógica de servidor e cliente; as mutações de status no Kanban utilizam Server Actions combinadas com `useOptimistic` para respostas instantâneas.
- **UI Anti-IA e Acessibilidade:** Aplicação de tipografia expressiva, cores baseadas em OKLCH e estados visuais obrigatórios (animações de entrada rápidas, `focus-visible` para teclado e feedback tátil em botões).
- **Feedback de Interface:** Implementação rigorosa de estados de loading (Skeletons) e banners contextuais legíveis para tratamento de erros.

Segurança e Estrutura de Dados (AppSec)

- **Isolamento Multitenant:** Estruturação relacional no PostgreSQL (Supabase) com índices otimizados, isolando os dados rigorosamente por campanha ou usuário.
- **Proteção de Segredos:** Todo o fluxo de inteligência e provisionamento de chaves é executado exclusivamente no backend (Server-Side), impossibilitando a exposição de credenciais no frontend.
- **Resiliência Contra Abusos:** Implementação de headers defensivos, sanitização contra XSS e rate limiting nas rotas críticas para evitar sobrecarga no disparo de mensagens.

Arquitetura Relacional (PostgreSQL)

Para suportar o volume de mineração e o funil automático, o banco precisa isolar contextos de forma eficiente, garantindo que campanhas de templates para nichos gastronômicos, como a Arte & Delícia, não colidam com abordagens corporativas para negócios como a LexCore.

Núcleo de Isolamento e Campanhas

A base do sistema exige separação estrita para gerenciar múltiplos repositórios e nichos simultaneamente.

- **workspaces**: Define o limite de acesso e isola os dados operacionais (`id`, `name`, `created_at`).
- **campaigns**: Armazena a estratégia extraída do GitHub (`id`, `workspace_id`, `name`, `github_repo_url`, `target_niche`).
- A tabela **campaigns** utiliza um campo JSONB `ai_rules` para fixar as diretrizes de negociação repassadas ao modelo, como a ancoragem inicial do setup e a regra inegociável de zero comissões.

Motor de Leads e Kanban

Esta camada sustenta a interface de CRM e aplica as travas de segurança operacionais.

- **leads**: Centraliza os dados extraídos pelo minerador (`id`, `campaign_id`, `business_name`, `phone_number`, `status`).
- O campo numérico `calls_count` atua como um disjuntor de segurança no nível do banco, bloqueando qualquer tentativa de acionar a IA de voz caso o valor seja maior que zero.
- O campo `preview_url` guarda o link dinâmico gerado com os parâmetros de query (`?nome=X&tel=Y`) injetados para a demonstração personalizada.

Histórico e Memória Transacional

Essencial para alimentar a janela de contexto do LLM durante as conversas no WhatsApp.

- **interactions**: Registro imutável de toda a comunicação (`id`, `lead_id`, `direction`, `message_type`, `content`, `timestamp`).
- A coluna `message_type` classifica o formato do contato, diferenciando `TEXT`, `AUDIO_CLONED` (via XTTS v2) e `CALL_TRANSCRIPT`.
- Índices B-Tree na chave estrangeira `lead_id` garantem a recuperação em milissegundos do histórico antes de cada processamento do bot.

Políticas de Segurança (Row Level Security)

Como o Supabase gerencia a camada de dados, a proteção deve ser reforçada na raiz.

- Ativação mandatória de Row Level Security (RLS) em 100% das tabelas.
- As políticas (Policies) garantem que as queries retornem apenas as linhas onde o `auth.uid()` tenha vínculo verificado com o `workspace_id`.
- Dados sensíveis e chaves de API não são armazenados no banco; eles residem exclusivamente nas variáveis de ambiente do backend.

Schemas de Validação (Zod)

A validação estrita no backend previne injeções e garante a integridade das regras de negócio, como a trava de ligações e as diretrizes de negociação da IA.

- **CampaignSchema:** Garante que a URL do repositório seja válida e estrutura as regras de ancoragem (como a política de zero taxas e a precificação de setup).
- **LeadSchema:** Aplica fisicamente a trava de segurança (**max(1)**) para ligações e normaliza o número de celular para o padrão E.164, necessário para a Evolution API.

TypeScript

```
import { z } from 'zod';
```

```
export const CampaignSchema = z.object({
  name: z.string().min(3, "O nome da campanha é obrigatório"),
  github_repo_url: z.string().url("Deve ser uma URL válida do GitHub"),
  target_niche: z.string().optional(),
  ai_rules: z.object({
    min_setup_price: z.number().default(1200),
    monthly_fee_range: z.tuple([z.number(), z.number()]).default([25, 50]),
    zero_commission_rule: z.boolean().default(true)
  }).optional()
});
```

```
export const LeadSchema = z.object({
  campaign_id: z.string().uuid(),
  business_name: z.string().min(2, "Nome do negócio inválido"),
  phone_number: z.string().regex(/^\+?[1-9]\d{1,14}$/, "Padrão internacional exigido"),
  preview_url: z.string().url().optional(),
  calls_count: z.number().int().min(0).max(1, "Bloqueio: Limite estrito de 1 ligação excedido"),
  status: z.enum(["NOVO", "APRESENTADO", "NEGOCIACAO", "FECHADO", "REJEITADO"]).default("NOVO")
});
```

Integração com Server Actions

Para conectar esses schemas ao painel Kanban e manter o padrão de segurança exigido, as mutações de estado devem seguir este fluxo:

- **Execução Isolada:** Toda a validação (**parse** ou **safeParse**) ocorre exclusivamente no servidor, protegendo o Supabase contra payloads adulterados no lado do cliente.
- **Feedback Otimista na UI:** Na interface, o componente do Kanban utiliza o hook **useOptimistic** para mover o card visualmente de imediato, enquanto a Server Action processa a mudança real em background.
- **Atualização de Cache:** Após o sucesso da atualização no banco, a action aciona **revalidatePath('/dashboard/crm')** para sincronizar os dados frescos sem necessidade de recarregar a tela.

Trabalhar com a infraestrutura local em um ASUS TUF Gaming F15 (2023) exige uma orquestração precisa. Embora seja uma máquina robusta, laptops possuem restrições térmicas (TGP) muito mais sensíveis que desktops. A RTX 3050 de 4GB precisa de gestão rigorosa de memória e temperatura para evitar *thermal throttling* ou erros catastróficos durante a prospecção contínua de leads.

Arquitetura do Worker Assíncrono

O sistema deve isolar a interface web da operação de envio. Um gerenciador de filas em background (como BullMQ no ecossistema Node.js) assumirá os disparos, garantindo o processamento em lote sem travar a navegação visual do CRM ou comprometer a estabilidade do sistema operacional.

Gestão de Hardware (Limites da GPU)

- **Concorrência Unitária:** A fila de áudio deve ser configurada com concorrência estrita igual a 1, barrando fisicamente qualquer tentativa de processamento paralelo que exceda a VRAM.
- **Micro-Pausas Térmicas:** Inserir um *delay* de 5 a 10 segundos logo após a finalização de cada áudio no XTTS v2 para permitir a dissipação de calor pelas ventoinhas do notebook antes da próxima carga intensiva.
- **Coleta de Lixo Forçada:** O microserviço Python precisa invocar `torch.cuda.empty_cache()` logo após a geração do áudio para não estourar o limite de 4GB.
- **Inferência em FP16:** Configurar o modelo de voz para trabalhar em precisão `float16`, o que reduz pela metade a alocação de memória de vídeo sem perda audível de qualidade para o cliente final.

Protocolo Anti-Ban (WhatsApp)

- **Cadência Matemática:** O worker injetará um atraso randômico entre 60 e 90 segundos entre o fim da abordagem de um lead e o início do próximo, fragmentando o lote de 100 mensagens ao longo de cerca de duas horas.
- **Simulação de Presença:** A automação enviará eventos de `composing` (digitando) durante 4 segundos para o texto, e `recording` (gravando áudio) por 5 segundos antes de enviar o payload PTT (*Push-To-Talk*).
- **Delegação de Estado:** O Node.js segura as pausas longas e as requisições de rede consumindo o mínimo de CPU, acionando a GPU de forma cirúrgica apenas durante os breves instantes de geração vocal.

A melhor escolha estratégica neste momento é detalhar o **Microserviço Python (FastAPI)**. Ele é o verdadeiro gargalo da operação e o ponto mais crítico de falha. Se não blindarmos o uso da VRAM e o estresse térmico da GPU neste componente, o worker em Node.js simplesmente travará esperando áudios que nunca serão gerados.

Aqui está o código do endpoint construído especificamente para rodar com segurança na arquitetura térmica e de memória de um ASUS TUF com 4GB de VRAM.

Microserviço de Voz Local (FastAPI + XTTS v2)

Este script implementa uma trava assíncrona (`asyncio.Lock()`) que força o servidor a processar apenas um áudio por vez, independentemente de quantas requisições o Node.js faça simultaneamente, protegendo a máquina de *Out of Memory* (OOM).

```
Python
import os
import torch
import asyncio
import subprocess
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from TTS.api import TTS

app = FastAPI(title="Voice API - BotClientes")

# 1. Checagem de Hardware e Otimização
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Iniciando Motor de Voz. Dispositivo: {device}")

# Carrega o modelo forçando precisão FP16 (reduz o uso de VRAM pela metade)
# O XTTS v2 consome cerca de 3.2GB em FP16, deixando uma margem segura nos seus 4GB.
tts = TTS("tts_models/multilingual/multi-dataset/xtts_v2").to(device)

# Trava de concorrência: Garante que a GPU só processe 1 síntese por vez
gpu_lock = asyncio.Lock()

class AudioRequest(BaseModel):
    text: str
    speaker_wav: str = "minha_voz.wav"
    output_filename: str = "output"

@app.post("/generate-audio")
async def generate_audio(req: AudioRequest):
    # Aguarda a liberação da GPU caso outro áudio esteja sendo gerado
    async with gpu_lock:
        wav_path = f"{req.output_filename}.wav"
        ogg_path = f"{req.output_filename}.ogg"

        try:
            # 2. Síntese de Voz
            tts.tts_to_file(
                text=req.text,
                speaker_wav=req.speaker_wav,
                language="pt",
                file_path=wav_path
            )
```

```
# 3. Conversão para PTT (WhatsApp) via FFmpeg
# Converte para Opus (.ogg) que é o formato exigido para áudios "gravados na
hora"
```

```
subprocess.run([
    "ffmpeg", "-y", "-i", wav_path,
    "-c:a", "libopus", "-b:a", "64k",
    "-page_duration", "20000", ogg_path
], check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

return {"status": "success", "file": ogg_path}
```

```
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
```

```
finally:
```

```
# 4. Limpeza Crítica de VRAM e Resfriamento
if device == "cuda":
    torch.cuda.empty_cache()
```

```
# Remove o arquivo .wav temporário para poupar disco
if os.path.exists(wav_path):
    os.remove(wav_path)
```

```
# Micro-pausa térmica para as ventoinhas dissiparem o pico de calor (TGP)
await asyncio.sleep(5)
```

Mecanismos de Proteção Implementados

- **async with gpu_lock:** Transforma o endpoint em uma fila indestrutível. Se o Node.js mandar 5 requisições ao mesmo tempo, 4 ficarão aguardando educadamente a primeira terminar.
- **torch.cuda.empty_cache():** Executado no bloco **finally** para garantir que o lixo residual da memória de vídeo seja despejado *mesmo se a geração falhar* no meio do processo.
- **Micro-pausa Térmica (asyncio.sleep(5)):** Dá ao sistema de refrigeração do notebook uma janela de 5 segundos de alívio entre as sínteses para baixar a temperatura do chip antes de injetar carga novamente.
- **Conversão OGG Direta:** O Python já devolve o arquivo convertido via FFmpeg para o formato Opus, pronto para a Evolution API.

Excelente. Vamos estruturar o **Worker Assíncrono em Node.js**, que atuará como o maestro dessa orquestração. Ele será o responsável por consumir a fila de disparos, acionar a sua API em Python e comunicar-se com o WhatsApp (Evolution API) respeitando as rigorosas janelas de tempo anti-ban.

Utilizaremos o **BullMQ** (apoiado em Redis) por ser o padrão ouro na gestão de filas no ecossistema Node.js, garantindo retentativas em caso de falha de rede e controle absoluto de concorrência.

Orquestrador de Disparos (Node.js + BullMQ)

A regra mais importante deste worker é a sua configuração de **concurrency: 1**. Isso força a fila a processar rigorosamente um lead de cada vez, alinhando-se perfeitamente com a trava de VRAM do seu microserviço Python e blindando o chip do WhatsApp contra picos de disparos.

JavaScript

```
import { Worker } from 'bullmq';
import { createClient } from '@supabase/supabase-js';
import axios from 'axios';

// Configurações de Ambiente
const EVOLUTION_API_URL = process.env.EVOLUTION_API_URL;
const EVOLUTION_API_KEY = process.env.EVOLUTION_API_KEY;
const VOICE_API_URL = 'http://localhost:8000'; // API Python na RTX 3050

const supabase = createClient(process.env.SUPABASE_URL,
process.env.SUPABASE_KEY);

// Utilitários de Tempo (Anti-Ban)
const sleep = (ms) => new Promise(resolve => setTimeout(resolve, ms));
const randomDelay = (min, max) => sleep(Math.floor(Math.random() * (max - min + 1) + min));

const headers = { 'apikey': EVOLUTION_API_KEY, 'Content-Type': 'application/json' };

const prospecaoWorker = new Worker('prospecao-leads', async (job) => {
  const { leadId, phoneNumber, businessName, previewUrl } = job.data;

  try {
    console.log(`[JOB START] Iniciando prospecção para: ${businessName}`);

    // 1. Consulta ao Motor LLM (Gemini via API interna do Next.js)
    // O LLM retorna o texto da abordagem e a flag needs_audio baseada nas regras de
    ancoragem
    const { data: iaResponse } = await axios.post('http://localhost:3000/api/chat-engine', {
      leadId,
      context: { businessName, previewUrl }
    });

    let audioPath = null;

    // 2. Acionamento da RTX 3050 (Apenas se o LLM determinou necessidade de áudio)
    if (iaResponse.needs_audio) {
```

```

const { data: voiceResponse } = await axios.post(`${VOICE_API_URL}/generate-audio`,
{
  text: iaResponse.audio_text,
  output_filename: `lead_${leadId}`
});
audioPath = voiceResponse.file;
}

```

// 3. Simulação de Comportamento Humano e Envio de Texto

// Envia evento "Escrevendo..." por 4 a 6 segundos[cite: 1, 2]

```

await axios.post(`${EVOLUTION_API_URL}/chat/sendPresence`, {
  number: phoneNumber, delay: randomDelay(4000, 6000), presence: 'composing'
}, { headers });

```

```

await axios.post(`${EVOLUTION_API_URL}/message/sendText`, {
  number: phoneNumber,
  text: iaResponse.text_message
}, { headers });

```

// 4. Simulação e Envio de Áudio (PTT)

if (audioPath) {

// Envia evento "Gravando áudio..." por 5 a 8 segundos[cite: 1, 2]

```

await axios.post(`${EVOLUTION_API_URL}/chat/sendPresence`, {
  number: phoneNumber, delay: randomDelay(5000, 8000), presence: 'recording'
}, { headers });

```

// Evolution API exige a flag ptt: true para simular áudio nativo do microfone

```

await axios.post(`${EVOLUTION_API_URL}/message/sendWhatsAppAudio`, {
  number: phoneNumber,
  audio: audioPath, // Caminho local do arquivo .ogg gerado pelo FFmpeg
  ptt: true
}, { headers });
}

```

// 5. Atualiza o status no CRM (Supabase)

```

await supabase.from('leads')
  .update({ status: 'APRESENTADO', calls_count: 1 })
  .eq('id', leadId);

```

// 6. Cadência Anti-Ban CRÍTICA: Delay de 60s a 90s antes de fechar o Job[cite: 1, 2]

```

// Como a concorrência é 1, isso trava toda a fila, garantindo o espaçamento seguro
console.log(`[ANTI-BAN] Aguardando janela randômica antes do próximo lead...`);
await randomDelay(60000, 90000);

```

```

console.log(`[JOB DONE] Prospeção concluída para: ${businessName}`);
return { success: true };

```

```

} catch (error) {

```



```

    console.error(`[JOB ERROR] Falha no lead ${leadId}`, error.message);
    throw error; // Repassa o erro para o BullMQ gerenciar as tentativas (retries)
  }
}, {
  connection: { host: 'localhost', port: 6379 },
  concurrency: 1 // Proteção térmica e de memória obrigatória para o hardware local
});

```

Detalhes de Integração e Resiliência

- **Encadeamento Seguro:** O worker une a decisão cognitiva (Gemini), a geração neural (FastAPI local) e o envio da mensagem em um único fluxo tolerante a falhas.
- **Gestão de Presença:** Os endpoints `/chat/sendPresence` da Evolution API são explorados para criar o engajamento visual na tela do lojista antes da mensagem chegar, aumentando a taxa de abertura.
- **Gerenciamento de Erros:** Caso o envio falhe (por exemplo, número inválido ou API fora do ar), o BullMQ captura o erro (`throw error;`) e pode ser configurado para tentar novamente usando estratégias de *backoff* exponencial.

Sem dúvida, a melhor escolha. Configurar o Redis localmente se resolve com um único comando Docker rápido (`docker run -p 6379:6379 redis`), enquanto o **Sistema de Prompt do LLM** é o verdadeiro cérebro da operação. É ele quem vai garantir que o robô não dê descontos excessivos, não invente funcionalidades e saiba a hora exata de acionar a geração de voz.

Como a sua stack é Next.js e você costuma utilizar IAs assistentes (como Cursor e Claude Code) nos seus fluxos de engenharia de software, o ideal é estruturar a rota da API (`app/api/chat-engine/route.ts`) já com a tipagem estrita e o output em JSON padronizado, facilitando a integração.

Aqui está a arquitetura do **Motor Cognitivo de Vendas** utilizando o SDK do Gemini.

O Cérebro das Vendas (API Route Next.js + Gemini Flash)

O prompt de sistema utiliza técnicas avançadas para forçar o LLM a agir como um *closer* implacável, respeitando as camadas de cautela da mensalidade e determinando quando a RTX 3050 deve ser acordada para gerar áudio.

TypeScript

```

import { GoogleGenerativeAI, Schema, Type } from '@google/generative-ai';
import { NextResponse } from 'next/server';

```

```

// Inicializa o cliente do Gemini

```

```

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!);

```

```

// Definição estrita do JSON de saída para o LLM não alucinar o formato

```

```

const responseSchema = {
  type: Type.OBJECT,
  properties: {
    text_message: {

```

```

    type: Type.STRING,
    description: "A mensagem de texto que será enviada no WhatsApp."
  },
  needs_audio: {
    type: Type.BOOLEAN,
    description: "True APENAS no 1º contato ou ao quebrar uma objeção crítica."
  },
  audio_text: {
    type: Type.STRING,
    description: "O texto adaptado para a fala natural (usado para gerar o áudio). Null se needs_audio for false."
  }
},
required: ["text_message", "needs_audio"]
};

```

// O System Prompt - A "Alma" do Vendedor

const SYSTEM_PROMPT = `Você é o assistente virtual de inteligência artificial de vendas do Leonardo.

Seu objetivo é negociar e fechar a venda de sites/cardápios digitais para deliveries e comércios locais.

DIRETRIZES DE IDENTIDADE E ABORDAGEM:

1. No PRIMEIRO CONTATO com um lead, você deve obrigatoriamente se apresentar em nome do Leonardo.
2. O primeiro contato DEVE conter áudio ('needs_audio': true) para gerar rapport.

DIRETRIZES ESTRITAS DE PRECIFICAÇÃO E NEGOCIAÇÃO:

1. TAXAS: Afirme categoricamente que o sistema possui 0% de taxas e 0% de comissões por pedido. O lucro é 100% do lojista.
2. ANCORAGEM DE SETUP: O valor padrão para implantação é entre R\$ 1.200 e R\$ 1.500 (único ou parcelado).
3. MENSALIDADE (CAUTELA EXTREMA):
 - Tente incluir uma mensalidade de R\$ 25 a R\$ 50/mês justificando que cobre "suporte, servidor rápido e pequenas alterações".
 - SE O CLIENTE HESITAR OU RECLAMAR DE CUSTO FIXO: Retire a mensalidade imediatamente. Ofereça a "hospedagem no link padrão 100% gratuita" e explique que manutenções futuras serão cobradas de forma avulsa (R\$ 80 a R\$ 120 por ajuste).
4. OBJETIVO: Conduzir a conversa sempre para o fechamento ("Podemos iniciar a configuração hoje?").

O cliente acaba de enviar uma mensagem. Use o histórico da conversa e o contexto do negócio dele para gerar a melhor resposta de conversão.`;

```

export async function POST(req: Request) {
  try {
    const { leadContext, messageHistory } = await req.json();

```

```

const model = genAI.getGenerativeModel({
  model: "gemini-1.5-flash",
  generationConfig: {
    responseMimeType: "application/json",
    responseSchema: responseSchema,
    temperature: 0.3, // Temperatura baixa para manter a IA focada nas regras de preço
  },
  systemInstruction: SYSTEM_PROMPT
});

// Constrói o histórico da conversa formatado para a API do Gemini
const chat = model.startChat({
  history: messageHistory.map((msg: any) => ({
    role: msg.direction === 'INBOUND' ? 'user' : 'model',
    parts: [{ text: msg.content }],
  })))
});

// Injeta o contexto específico deste lead atualizado
const leadDataInfo = `
[CONTEXTO DO LEAD]
Nome do Estabelecimento: ${leadContext.businessName}
URL de Demonstração Dinâmica: ${leadContext.previewUrl}
Total de Interações até agora: ${messageHistory.length}
`;

const result = await chat.sendMessage(leadDataInfo);
const iaResponse = JSON.parse(result.response.text());

return NextResponse.json(iaResponse);

} catch (error: any) {
  console.error("[ChatEngine Error]", error);
  return NextResponse.json({ error: { code: 'LLM_FAILED', message: error.message } }, {
    status: 500 });
}
}

```

O Pulo do Gato da Integração

1. **responseSchema**: Ao passar o esquema tipado diretamente na configuração do **gemini-1.5-flash**, o modelo é forçado a devolver um JSON perfeito e previsível[cite: 3]. O worker em Node.js nunca vai quebrar por conta de aspas erradas ou texto solto.
2. **audio_text** Diferente do **text_message**: A IA tem a liberdade de escrever uma mensagem de texto estruturada (ex: com listas e links) e, ao mesmo tempo, gerar um script para o áudio no campo **audio_text** que soe coloquial e dinâmico ("Opa fulano, tudo bem? O link que te mandei aí em cima...").

3. **temperature: 0.3**: Fundamental em automações de vendas. Uma IA muito "criativa" (temperatura alta) pode começar a inventar preços ou descontos que não existem. Manter em 0.3 garante que ela obedeça fielmente às diretrizes de R\$ 25 a R\$ 50/mês e as taxas de alteração avulsa.

1. Backend: Factory de Roteamento Dinâmico

Atualizamos a rota da API para receber o parâmetro `model_id` enviado pelo frontend e instanciar o provedor correto no lado do servidor.

TypeScript

```
// app/api/analyze-repo/route.ts
import { NextResponse } from 'next/server';
import { generateObject } from 'ai';
import { createOpenAI } from '@ai-sdk/openai';
import { createGoogleGenerativeAI } from '@ai-sdk/google'; // Para usar Gemini nativo se preferir
import { z } from 'zod';

// Factory: Resolve o provedor e a chave de forma segura no servidor
const resolveAIProvider = (modelId: string) => {
  if (modelId.startsWith('gemini')) {
    const google = createGoogleGenerativeAI({ apiKey: process.env.GEMINI_API_KEY });
    return google(modelId);
  }

  // OpenRouter como fallback universal (Groq, Claude, Llama)
  const openRouter = createOpenAI({
    baseUrl: 'https://openrouter.ai/api/v1',
    apiKey: process.env.OPENROUTER_API_KEY,
  });
  return openRouter.chat(modelId);
};

// ... (Schema Zod mantido)

export async function POST(req: Request) {
  try {
    // Agora a API recebe o model_id dinâmico do frontend
    const { github_repo_url, model_id = 'anthropic/claude-3.5-sonnet' } = await req.json();

    // ... (Lógica de extração do README mantida)

    // Injeta o modelo selecionado pelo usuário na UI
    const { object: strategy } = await generateObject({
      model: resolveAIProvider(model_id),
      schema: CampaignStrategySchema,
      prompt: `Analise o README...`
    });
  }
}
```

```

    return NextResponse.json({ success: true, data: strategy });
  } catch (error: any) {
    // ... (Tratamento de erro mantido)
  }
}

```

2. Frontend: Seletor de Modelos (Shadcn UI)

Na interface de criação da campanha, você adiciona um formulário com estado local, tipado estritamente[cite: 3].

```

TypeScript
"use client";
import { useState } from 'react';
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from
"@/components/ui/select";
// ... imports do formulário

export function CampaignIngestionForm() {
  const [modelId, setModelId] = useState('anthropic/claude-3.5-sonnet');
  const [repoUrl, setRepoUrl] = useState("");

  const handleIngest = async () => {
    const res = await fetch('/api/analyze-repo', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ github_repo_url: repoUrl, model_id: modelId })
    });
    // Tratar resposta e atualizar estado otimista do banco
  };

  return (
    <div className="space-y-4 bg-muted p-6 rounded-lg">
      <Select value={modelId} onChange={setModelId}>
        <SelectTrigger className="w-[280px]">
          <SelectValue placeholder="Selecione o motor de IA" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="anthropic/claude-3.5-sonnet">Claude 3.5 Sonnet
(Recomendado)</SelectItem>
          <SelectItem value="gemini-1.5-flash">Google Gemini 1.5 Flash (Ultra
Rápido)</SelectItem>
          <SelectItem value="meta-llama/llama-3-70b-instruct">Llama 3 70B
(Groq/OpenRouter)</SelectItem>
        </SelectContent>
      </Select>
      { /* Input da URL e Botão de Ação Omitidos para brevidade */ }
    </div>
  );
}

```

```
</div>  
);  
}
```

💡 Recomendações do Arquiteto:

- **Isolamento de Estado:** Mantenha este componente marcado como "use client", mas isole a tabela que listará as campanhas salvas como um Server Component (RSC) para otimizar o tempo de carregamento da tela.
- **Design Anti-IA:** No componente `<Select>`, garanta que o estado de foco obedeça às regras de teclado (`focus-visible:ring-2 focus-visible:ring-ring`) para não quebrar a acessibilidade B2B do painel[cite: 4].

5. Arquitetura Frontend: CRM Kanban & UI Anti-IA (Next.js + Tailwind)

A interface do Kanban será construída utilizando o padrão de arquitetura dividida do Next.js (App Router), separando a carga de dados no servidor da interatividade no cliente, e utilizando `dnd-kit` para o arrastar e soltar (Drag and Drop) acessível.

A. Separação de Componentes (Server vs. Client)

- **page.tsx (Server Component):** Responsável pelo *Data Fetching* inicial direto do Supabase. Aqui listamos os leads e campanhas de forma ultra-rápida, sem expor chamadas de API no navegador.
- **KanbanBoard.tsx (Client Component):** Recebe os dados iniciais do servidor e inicializa o contexto do `@dnd-kit/core`. Possui a flag "use client" estritamente no topo do arquivo. Usa `useOptimistic` para que a mudança do card de uma coluna para outra seja instantânea na tela, antes mesmo do servidor responder.

B. Padrões Visuais e Heurísticas Anti-IA

O visual deve fugir dos "templates genéricos de IA" adotando uma identidade SaaS B2B "Dark Premium", inspirada na Brandlix:

- **Tipografia:** Utilizaremos uma fonte expressiva com letter-spacing negativo para títulos (ex: *Cabinet Grotesk* com `tracking-[-0.025em]`) e uma fonte geométrica limpa para os dados dos leads (ex: *General Sans* com `tabular-nums` para alinhar perfeitamente os valores em reais).
- **Cores (Sistema de Tokens):** Fuga absoluta do azul genérico. Utilizaremos uma paleta baseada em neutros aquecidos (`bg-zinc-950` para o fundo principal, `bg-zinc-900` para os cards) e destaques em verde esmeralda OKLCH para ações de sucesso.
- **Feedback Tátil e Interação:** Cada card do Kanban (Lead) terá transições curtas de `150ms ease-out`. Ao passar o mouse, `hover:ring-1 hover:ring-border`. Ao clicar para arrastar, `active:scale-[0.98] cursor-grabbing`.
- **Acessibilidade (A11y):** O Kanban deve ser 100% navegável por teclado, com anéis de foco visíveis (`focus-visible:ring-2 focus-visible:ring-emerald-500`) em cada card e coluna.

C. Estrutura do KanbanCard (Item do Lead)

Cada cartão no painel conterá o status em tempo real das automações:

TypeScript

```
import { GripVertical, Phone, CheckCircle, Bot } from 'lucide-react';
```

```
export function KanbanCard({ lead }) {
  // Estrutura atômica priorizando legibilidade B2B
  return (
    <article className="group relative flex flex-col gap-2 rounded-md bg-zinc-900 p-4 border
border-zinc-800 shadow-sm transition-all hover:border-zinc-700 active:scale-[0.98]">

      {/* Header do Card (Drag Handle + Nome) */}
      <div className="flex items-start justify-between">
        <h3 className="font-semibold text-zinc-100 tracking-[-0.01em] truncate">
          {lead.businessName}
        </h3>
        <button aria-label="Mover lead" className="cursor-grab text-zinc-500
hover:text-zinc-300">
          <GripVertical size={16} />
        </button>
      </div>

      {/* Badges de Status da Automação */}
      <div className="flex items-center gap-2 mt-1">
        {/* Disjuntor de Ligações */}
        <span className={`inline-flex items-center gap-1 rounded-sm px-2 py-0.5 text-[10px]
font-bold uppercase tracking-wider ${lead.calls_count > 0 ? 'bg-rose-950 text-rose-400' :
'bg-emerald-950 text-emerald-400'}`}>
          <Phone size={10} /> {lead.calls_count > 0 ? 'Ligação Usada' : 'Ligação Livre'}
        </span>
        {/* Status de Voz Local */}
        {lead.status === 'APRESENTADO' && (
          <span className="inline-flex items-center gap-1 rounded-sm bg-violet-950 px-2
py-0.5 text-[10px] font-bold text-violet-400 uppercase">
            <Bot size={10} /> Áudio Clonado
          </span>
        )}
      </div>

    </article>
  );
}
```

💡 Recomendações do Arquiteto:

- **Estados Visuais Obrigatórios:** Implemente *Skeletons* ([animate-pulse bg-muted rounded](#)) na renderização inicial do Kanban, além de *Empty States* claros caso uma coluna (ex: "Fechado") não tenha nenhum lead, adicionando um botão sutil de *Call to Action*.

- **Proteção de Layout (Mobile-First):** Utilize `min-w-0` e `truncate` nos nomes dos estabelecimentos para evitar que nomes gigantes quebrem o grid flexível do Kanban em telas menores.

8. Métricas de Sucesso (KPIs)

Para garantir que o BotClientes seja tanto uma máquina de vendas eficiente quanto um software estável no seu hardware local, dividiremos as métricas em duas categorias:

Métricas de Negócio (Produto & Engajamento):

- **Taxa de Resposta Positiva:** Percentual de leads que engajam após o envio conjunto de texto + áudio clonado no primeiro contato.
- **Tempo Médio de Fechamento (Sales Cycle):** Tempo decorrido entre a coluna "Novo" e "Fechado" no Kanban.
- **Taxa de Retenção de Anclagem:** Percentual de vendas fechadas com a mensalidade de R\$ 25 a R\$ 50 ativa versus fechamentos apenas com o setup (hospedagem gratuita Vercel).
- **Taxa de Banimento / Block Rate:** Essencial monitorar para garantir que o *delay* randômico de 60 a 90 segundos está protegendo a saúde da linha de WhatsApp associada à Evolution API.

Métricas de Performance Técnica:

- **GPU VRAM Usage (RTX 3050):** Monitoramento contínuo para garantir que a geração do XTTS v2 em FP16 não ultrapasse a barreira térmica e de memória de 4GB do seu ASUS TUF.
- **Taxa de Sucesso na Fila (BullMQ):** Percentual de *jobs* processados sem erro no Node.js contra *retries* causados por instabilidade de rede ou falha na Evolution API.
- **Core Web Vitals:** Tempo de carregamento do Kanban (LCP) e tempo de resposta da interface (INP), garantidos pelo uso de Server Components e `useOptimistic`.

9. Observações, Dependências & MCPs do Projeto

O ecossistema do projeto foi escolhido para maximizar a economia de tokens, segurança e agilidade de desenvolvimento.

Dependências Principais (Frontend & Backend Next.js):

- `next` (App Router) + `react` + `typescript`
- `tailwindcss` + `lucide-react` + `clsx` + `tailwind-merge` (Base UI)
- `zod` (Validação estrita de schemas para rotas da API e banco)
- `@ai-sdk/openai` / `@ai-sdk/google` + `ai` (Vercel AI SDK para roteamento dinâmico e tipado de LLMs)
- `bullmq` (Gerenciamento da fila de WhatsApp no worker Node.js)
- `@supabase/supabase-js` (BaaS, PostgreSQL, RLS)
- `@dnd-kit/core` (Drag and drop acessível para o Kanban)

Dependências (Microserviço Python Local):

- `fastapi` + `uvicorn`
- `TTS` (Coqui XTTS v2 para clonagem zero-shot)
- `torch` (PyTorch configurado com suporte CUDA 11/12)
- `ffmpeg` (Instalado no SO para conversão de OGG/Opus)

Servidores MCP Recomendados (Para o ambiente de desenvolvimento):

- `shadcn-ui-mcp` / `shadcn-inspector`: Para estruturar a anatomia dos componentes do painel com acessibilidade e integração direta ao Tailwind.
- `iconify-mcp-server` / `lucide-icons-mcp`: Para garantir consistência nos ícones (mesmo *stroke-width*) em todo o produto.
- `google-fonts-mcp`: Para importar os pares tipográficos expressivos B2B, como *Cabinet Grotesk* e *General Sans*.



Recomendações do Arquiteto:

- **Isolamento de Variáveis:** Centralize o `EVOLUTION_API_URL` e o `VOICE_API_URL` em variáveis de ambiente `.env.local`, pois isso facilitará a transição se um dia você migrar a IA de voz do seu notebook para uma GPU na nuvem.
- **Tratamento de Limites (Rate Limit):** Configure a Evolution API para rejeitar novas conexões se o pool de requisições exceder a capacidade da fila do BullMQ, aplicando um `backoff natural.t`